Sequence-Level Knowledge Distillation for SLURP Command Parsing

CS3081 Cloud Computing Mini Project — Report

Pawan Kumar Dangi (230103011)
Supervisor: Dr. Salam Michael Singh

November 2026

Summary.

- Teacher Phi-3.5-mini-instruct (3.8B) labels 3,000 SLURP commands; student EleutherAI/pythia-70m (70M) is fine-tuned on the first 1,500 cleaned labels and served as FastAPI on a local k3d cluster with SeaweedFS.
- Student: 100% JSON validity, full-frame 29/100 — beats the teacher (28/100) on every metric.
- All evidence is command output in `results/`. Repo: https://github.com/pwnFirstGit/cloud-mini-project.
- Disclosure (§1–2): labelling and training ran on Kaggle GPU (same model, prompt, temperature 0, seed 42); repo code is CPU-based and `make train` runs on CPU.

1 Task 1 — Teacher labelling and cleaning

Setup

- Cluster `cs3081` from `cluster/k3d.yaml`; SeaweedFS in namespace `data`; buckets `datasets` and `models`.
- `commands_pool.jsonl` (3,000 commands, C0001–C3000) uploaded to `s3://datasets/`.
- Prompt in `prompts/teacher_prompt.txt`: exactly four keys (`intent, date, time, place`), eight intents, copy-exactly slots, shortest phrase, `null` if unmentioned, never invent, output only JSON.
- Worker (`worker/`): reads `SHARD_INDEX` / `SHARD_COUNT`; takes lines whose 0-based line number mod $2 = i$; writes {`id, text, teacher_raw`} to `shard-`i`.jsonl` in `s3://datasets/teacher/`.
- Sharding: embarrassingly parallel, deterministic, no coordination; each shard is an independent S3 object; `clean.py` merges both shards in id order, so re-running it reproduces the same checksum.

Labelling and cleaning

- In-cluster demo (40 commands): both demo Jobs completed 1/1 (2m57s, 3m7s), both pods ran together, shards 5,678 and 9,286 bytes, 20 lines each.
- Full labelling on Kaggle GPU (T4×2) via Hugging Face transformers: v2 (chat template, stop tokens, batch 8, max 128 tokens) finished 3,000/3,000 in 19.6 min (≈0.4 s/line); verification: 3,000 unique ids, none missing or extra.

Input	(a) invalid JSON	(b) wrong keys	(c) bad intent	(d) bad slot	duplicates	Kept
3,000	869	0	1	26	0	2,104

Table 1: `make clean` counts, which balance: $869 + 0 + 1 + 26 + 0 + 2{,}104 = 3{,}000$. `train.sha256` is `5e1393...ae4e3` (full hash in the file).

Issues and corrections

- **Ollama workers failed 6/6** with HTTP 500 under memory pressure. Correction: moved labelling to Kaggle GPU (HF transformers); model, prompt, greedy decoding, temperature 0 and seed 42 unchanged — a serving-layer change only.
- **Labelling v1 was slow** (≈13 s/line, model never stopped). Correction: v2 added chat template, stop tokens, batch 8, max 128 tokens — 19.6 min for 3,000 lines.

- **869 rejections** are valid JSON wrapped in `json` markdown fences despite the no-markdown instruction. Correction: rejected under rule (a), teacher output not edited or recovered; training not blocked (2,104 > 1,500).

2 Task 2 — Training and evaluation

Training

- Student: EleutherAI/pythia-70m — suitable: small English causal LM, short JSON outputs, fits CPU memory; not instruction-tuned, so fine-tuning was required.
- Config (`training/config.yaml`): seed 42 (v2: seed 43), learning rate 2×10^{-4}, batch 8, epochs 3, max sequence length 256, first 1,500 cleaned rows, format `Command: <cmd> JSON: <teacher JSON>`.
- Library versions: torch 2.5.1, transformers 4.47.1, datasets 4.8.5, accelerate 1.15.0, Python 3.13.16.
- Loss (`training/loss.csv`, one value per epoch): v1 1.586 / 0.911 / 0.655; v2 1.591 / 0.900 / 0.649.
- `make train SUBSET=50` verified on Mac CPU in 705 s (21 steps).
- Models in S3: `s3://models/v1/` and `s3://models/v2/`; v1 also a GitHub Release asset, `models/v1/model.sha2` = `ae0f8c16...e08d` (full hash in the file).

Evaluation

Model	JSON validity	Intent	Date	Time	Place	Full-frame
Teacher (Phi-3.5)	64	49	49	48	53	28
Untrained pythia-70m	0	0	0	0	0	0
Student v1 (trained)	100	67	72	82	77	29

Table 2: Validation (100 rows, `score.py`). Quality gate passes: 100 ≥ 95. Hidden-test gate passes: student full-frame 29 ≥ 14 (half the teacher's 28).

- Student beats the teacher on every metric; raw outputs and score reports for all three models are in `results/` (teacher's as `results/teacher_validation.json`).

Distillation explanation

- **Sequence-level:** student is trained on the teacher's generated sequences (the full JSON output) as targets.
- **Logit-based:** student is trained to match the teacher's token-level output distribution (soft probabilities over the vocabulary at each step).
- **Why logit-based is impossible here:** Ollama (and the HF text-generation path used) returns text only, no teacher logits.
- **One real limitation:** student only ever sees the teacher's greedy outputs, so it inherits the teacher's errors and formatting quirks and cannot recover token-level uncertainty.

Issues and corrections

- **Training v1 loss was broken** (epoch 1: 141.55, epochs 2–3: exactly 0.0; transformers 5.20.2 fault) and eval was 0% on every metric. Correction: pinned torch 2.5.1 + transformers 4.47.1, masked pad labels with -100 — loss became sane (3.23 → 0.58).
- **Eval still 0% despite good loss.** Root cause: greedy regex {`.*`} in `predict.py` grabbed through the last }, including garbage after the valid JSON (model never learned EOS). Correction: non-greedy {`.*?`} in `predict.py` and `service/app.py` — student then scored 100/67/72/82/77/29.
- `make train SUBSET=50` **loss on Mac was not credible** (3263 → 0, grad. norm NaN) because the Mac has transformers 5.20.2; repo pins 4.47.1, which gives the sane losses above.

3 Task 3 — Service

Endpoints and configuration

- FastAPI, port 8000, `service/`: `POST /parse` returns only the 4-key JSON, with headers `X-Model-Version` and `X-Served-By` (pod name).
- `GET /healthz` returns 200; `GET /readyz` returns 503 until the model is loaded, 200 after.
- Config only from `MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT`, `MAX_NEW_TOKENS`=64, plus `FAULT_INJECT`.
- Model + tokenizer downloaded from S3 at startup; credentials only from a Kubernetes Secret; no weights in the image.
- Schema check against `schema.json`: invalid output returns HTTP 500 with exactly {`"error": "invalid_model_output"`}; never broken JSON with HTTP 200.
- Image `student-service:v1`: 1.27 GB (<2 GB, CPU-only torch); every dependency version pinned.

Tests

- 50 sequential requests at 1 CPU / 1 GiB: 50/50 HTTP 200, peak memory 767,803,392 bytes (≈732 MiB, `results/task3_50req.txt`).
- `FAULT_INJECT=1`: `/parse` returned HTTP 500 with exactly that body and both headers (`results/task3_fault_i`
- Cold-start log proves the S3 download: `Downloading model from s3://models/v1 via http://seaweedfs.d` and `Downloaded model files from s3://models/v1 to /tmp/model` (`results/task4_coldstart_download`

Issues and corrections

- **`/parse` returned 500 for every command** at first. Root cause: Mac copy of `service/app.py` still had the greedy regex. Correction: fixed to non-greedy, rebuilt — `/parse` 200.
- **Headers were missing.** Root cause: headers were set on the injected `response` param, lost when returning a `JSONResponse`. Correction: pass `headers=` to the returned `JSONResponse`.
- **First 50-request test was invalid** (peak 874 MiB against an unlimited container). Correction: re-ran at 1 CPU / 1 GiB — 50/50, peak 732 MiB.
- **Logs showed no model download.** Correction: added print lines in `_download_model()` — logs now show the S3 download explicitly.

4 Task 4 — Deployment and reliability

Deployment

- Namespace `app`: Deployment `parser` (2 replicas, requests/limits 1 CPU / 1 GiB, liveness `/healthz`, readiness `/readyz`), Service, Ingress `/parse` (Traefik), HPA min 1, max 2, target 70% CPU.
- **Limitation:** single-node laptop cluster proves nothing about multi-machine fault tolerance; all claims below are about this node only.

Reliability tests

- **Rolling update v1→v2** under 60-request load: 60/60 succeeded, 0% failed, 479 s, `X-Model-Version: v2` confirmed (`results/task4_update_load.txt`).
- **Rollback** (`kubectl rollout undo`): 60/60, 0% failed, 485 s, `X-Model-Version: v1` confirmed (`results/task4_rollback_load.txt`).
- **HPA:** scaled 1→2 under load (42%/70%, second pod Running) and back to 1 at 0% CPU; HPA load 60/60, 0%, 436 s.
- **Delete-a-Pod:** replacement 1/1 Running in ≈2 min while the surviving pod kept serving; load 60/60, 0%, 435 s (≤2% target).
- **Cold start** (pod creation → Ready, from pod timestamps): 8.0, 8.0, 8.0 s, median **8.0 s**; logs show model download (see Task 3).
- **Load balancing** over 60 requests: replica 1 served 33 (55.0%), replica 2 served 27 (45.0%), both ≥30%.

Issues and corrections

- **Image mismatch:** `deploy/app.yaml` expects `parser:v1`, service image was `student-service:v1`. Correction: `docker tag student-service:v1 parser:v1` before import.
- **Missing secret:** `s3-creds` was only in the pipeline namespace. Correction: created the secret in namespace `app`.
- **Rollouts stalled on Insufficient memory** (Docker limited to 3 GB; two 1 GiB pods plus SeaweedFS leave no room for a third pod). Correction: deleted an old pod manually to free memory — a limitation of this machine, not of the rollout mechanism.
- **First rolling-update probe (200 requests) was killed** (single `/parse` took 8.6 s, ≈30 min for 200). Correction: redone with 60 requests — 60/60, 0% fail.
- **Cold-start run 2 script crashed** (Mac/node clock-filter bug). Correction: recovered its time (8.0 s) from pod timestamps; run 3 completed normally.

5 Task 5 — Subnets and NACL

Subnets

- From 10.20.0.0/16, largest first, smallest prefix that fits: app 10.20.0.0/22 (usable 10.20.0.1–10.20.3.254, 1,022 hosts); public 10.20.4.0/24 (254); data 10.20.5.0/25 (126); management 10.20.5.128/27 (30). Full table with broadcast addresses in `network/subnets.md`.

NACL rules

- Inbound: rule 100 denies 198.51.100.0/24 before rules 120/130 allow TCP 80/443 from anywhere; TCP 22 only from the management subnet (rule 110); ephemeral replies (1024–65535) only from the app subnet (rule 140).
- Outbound: TCP 8000 only to the app subnet (rule 110); rule 115 denies all other traffic to 10.20.0.0/16; ephemeral replies to clients (rule 120); 198.51.100.0/24 denied outbound at rule 100.
- Check: `nacl_sim.py --check` on the six sample packets passes 6/6 (100%).

Explanation

- A stateless NACL has no memory of connections, so unlike a stateful security group it cannot infer that a reply belongs to an allowed request — every packet, including replies, is checked on its own, so reply traffic needs its own rules (ephemeral ports 1024–65535).
- Rule order matters: lowest number is checked first, so the deny for 198.51.100.0/24 (rule 100) fires before the allow rules for 80/443 (120/130) — a deny only works if its number is lower than the allow it must override.

Issues and corrections

- **First NACL missed the outbound deny** for the rest of 10.20.0.0/16. Correction: added rule 115 (deny all to 10.20.0.0/16) between the 8000 allow and the reply allow; re-checked 6/6.

6 Task 6 — CI/CD

Pipeline

- Repo: https://github.com/pwnFirstGit/cloud-mini-project (branch `main`); `AI_USE.md` records AI use.
- GitHub Actions, in order: lint (ruff, pyflakes only) → unit tests (8 tests) → model evaluation (downloads `model-v1.tar.gz` from Release v1, generates predictions, scores with `score.py`) → quality gate (`score.py --min-validity 95`, separate stage) → gitleaks scan.
- Release v1: `model.safetensors` (268.66 MiB, sha256 `ae0f8c16...e08d`, matching `models/v1/model.sha256`) and `model-v1.tar.gz` (216.45 MiB, sha256 `bf727d16...3aab`).

Results

- Passing run: https://github.com/pwnFirstGit/cloud-mini-project/actions/runs/42517471730 (2m54s) — evaluation wrote 100 predictions, JSON validity 100.0, intent 67, date 72, time 82, place 77; gate passed, gitleaks reported nothing.
- Deliberate failing run: https://github.com/pwnFirstGit/cloud-mini-project/actions/runs/42519101278 (branch `ci-fail-demo`, 3m0s) — untrained pythia-70m scores 0% validity, gate fails as intended.
- `make up` verified from the repo: creates cluster, SeaweedFS, builds/imports image, creates `app` namespace and `s3-creds` secret, uploads models to S3, applies `deploy/app.yaml`, rollout succeeds with 2 replicas; `/parse` then 200 with `X-Model-Version: v1`. `make down` deletes the cluster.

Issues and corrections

- **CI lint failed twice:** first 12 errors (import sorting, unused noqa, etc.), then 17 E501 line-length errors. Correction: `ruff check --select F` (pyflakes only).
- **Release upload timed out** (HTTP 408). Correction: retry succeeded; checksums verified to match `models/v1/model.sha256` exactly.
- `make up` **failed at** `kubectl wait` ("no matching resources found" — pod did not exist yet). Correction: use `kubectl rollout status deployment/seaweedfs`.
- `make up` **was incomplete** (no image build/import, no secret, no model upload). Correction: added build/tag/import, secret via dry-run apply, and `scripts/setup_s3.sh` (also uploads `commands_pool.jsonl` to `s3://datasets/`).

7 Task 7 — Measurement, cost and SLO

Load test

- Locust through the Ingress (`http://localhost:8080`, 4 users, 2 min).
- Run 1 (1 pod initially) failed: 19 requests, 16 failed (84%), p50 30,000 ms, 12× HTTP 0 (30 s timeout) and 4× HTTP 502; second pod scaled up mid-run (992m CPU); one pod restarted once.
- Run 2 (2 pods Ready first) — the reported standard run: 30 requests, 0 failed, average 15,248 ms, min 6,492 ms, max 20,580 ms, median 16,000 ms, throughput 0.25 req/s, p50 16,000 ms, p95 20,000 ms (`results/loadtest2_stats.csv`); per-replica 16 and 14 requests.
- `kubectl top pods` during run 1: 992m CPU / 718 MiB and 240m / 496 MiB for the two pods.
- Cold start (Task 4): median 8.0 s.

Cost

- From `rate_card.json` (Rs. 3.50/vCPU-hour + Rs. 0.45/GiB-hour, billed on limits): 2 replicas × (1 CPU × 3.50 + 1 GiB × 0.45) = Rs. 7.90/hour.
- At 0.25 req/s = 900 req/hour, cost per 1,000 requests = **Rs. 8.78**.

SLO

- **SLO:** 95% of `/parse` requests return 200 within 20 s in any 2-minute window.
- **Error budget:** 5% of requests (1.5 per 30-request window). Observed: 100% success, p95 exactly 20 s — SLO met, budget unused.
- **Issue:** p95 20 s exceeds the 5 s target in the assignment's Task 7 marking. Cause: CPU-bound, 64 tokens generated per request at 1 CPU. Improvement: reduce `MAX_NEW_TOKENS`, raise CPU limit, faster inference.

Review table

Component	Evidence (measured)	Limitation found	Improvement proposed
Data and storage	3,000 labelled; 2,104 kept; train.sha256 5e1393. . . ae4e3	869 (29%) rejected as fenced JSON	Stricter prompt / stop tokens; validate format at labelling time
Model quality	Student full-frame 29 vs teacher 28; validity 100 vs 64	Intent 67, date 72 — a third of frames still wrong	More training data; larger student; better teacher prompt
Latency and throughput	p50 16,000 ms, p95 20,000 ms, 0.25 req/s	p95 20 s exceeds the 5 s target; CPU-bound, 64 tokens	Reduce MAX_NEW_TOKENS; raise CPU limit; faster inference
Resource utilisation	Peak 732 MiB at 1 GiB limit; 992m CPU under load	3 GB Docker: no room for a third pod during rollout	Raise Docker memory; use maxSurge 0
Reliability and recovery	Cold start median 8.0 s; delete-a-pod load 60/60, 0% fail	Single node: no multi-machine fault tolerance	Multi-node cluster; pod disruption budgets
Security and cost	Cost Rs. 8.78 per 1,000 req; NACL 6/6	Test credentials in Secret; no TLS or auth on /parse	Real secrets via vault; TLS on Ingress; API auth

Table 3: Review table, as specified in the assignment: every Evidence cell carries a measured number.

Conclusion.

- All seven tasks are complete with measured evidence; student beats its teacher; service and deployment behave as specified on one node; CI gates the model.
- Main limits: single-node fault tolerance, 3 GB Docker memory (rollouts need manual pod deletion), CPU-bound latency (≈15 s p50 under load).